LangChain RAG Cheat Sheet

Naresh Edagotti
@Statfusionai

Your Quick Reference for Retrieval-Augmented Generation

DATA LOADING

- Import: PyPDFLoader, TextLoader, CSVLoader
- Load documents from various file formats

```
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("file.pdf")
documents = loader.load()
```

✂ DOCUMENT CHUNKING

- Import: RecursiveCharacterTextSplitter
- Split documents into smaller chunks

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
  chunk_size=500,
  chunk_overlap=50
)
chunks = splitter.split_documents(documents)
```

EMBEDDINGS

- Import: OpenAIEmbeddings, HuggingFaceEmbeddings
- Convert text to vector representations

```
from langchain.embeddings import OpenAIEmbeddings
embeddings = OpenAIEmbeddings()
```

VECTOR STORES

- Chroma, FAISS, Pinecone, Weaviate
- Store and retrieve vector embeddings

```
from langchain.vectorstores import FAISS
vectorstore = FAISS.from_documents(
  chunks, embeddings
)
```

RETRIEVAL

- Convert vector store to a retriever
- Retrieve relevant documents

```
retriever = vectorstore.as_retriever(
  search_type="similarity",
  k=3
)
docs = retriever.invoke("What is LangChain?")
```

RAG PIPELINE

- Import: RetrievalQA
- Combine retrieval and LLM for QA

```
from langchain.chains import RetrievalQA
qa = RetrievalQA.from_chain_type(
  llm,
  retriever=retriever,
  chain_type="stuff"
)
result = qa.run("Explain RAG")
```

EVALUATION

- Ragas for RAG evaluation
- LangSmith for tracing/debugging

```
# Evaluate RAG performance
# Use Ragas metrics for assessment
```

⚙ CUSTOMIZATION

- Custom retriever logic
- PromptTemplate for better responses

```
from langchain.prompts import PromptTemplate
prompt = PromptTemplate.from_template(
  "Answer using context: {context}\nQuestion: {question}"
)
```

Quick reference for LangChain RAG implementation